6-4 · CDC(Debezium) 실시간 캡처 · 이론 1H

CDC 개념과 Debezium 아키텍처

폴링 대신 binlog 를 읽는다 — 앱 수정 없이

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① 폴링 대신 로그 기반 CDC 를 쓰는 이유 두 가지를 말할 수 있다

② binlog(ROW 포맷)가 CDC 의 원천임을 설명한다

③ 스냅숏(op=r) 뒤 스트리밍(c/u/d)으로 이어지는 순서를 안다

④ 이벤트의 before/after/op 구조와 토픽 이름 규칙을 읽는다

다음 2.5H 실습에서 네 가지 op 를 전부 눈으로 확인합니다

1. CDC 란

CDC — 회의록을 받아 보는 방식

앱	MySQL binlog	Debezium	Kafka 토픽	받는 쪽
평소처럼 INSERT·UPDATE·DELETE	모든 변경을 순서대로 기록(ROW)	binlog 를 읽어 이벤트로	shop.shop.orders 테이블마다 하나	MERGE 로 멱등 적재(6-5)

애플리케이션 코드를 한 줄도 고치지 않는다 — DB 가 이미 남기는 기록을 가져올 뿐

참석자에게 매번 묻는 대신 서기(binlog)가 적어 둔 기록을 그대로 받는다 — Debezium 이 그 전달자

폴링과 무엇이 다른가 — 여섯 항목 비교

	폴링(주기적 SELECT)	로그 기반 CDC
방법	updated_at 으로 달라진 행 조회	binlog 를 읽음
DB 부담	매번 조회 — 부담이 큼	로그만 읽음 — 부담이 적음
DELETE	놓친다(행이 사라져서)	잡는다
중간 변경	놓친다(마지막 상태만)	전부 잡는다
지연	주기만큼(분 단위)	거의 실시간
앱 수정	updated_at 컬럼이 필요	필요 없음

DELETE 를 놓치는 것이 폴링의 결정적 약점 — 행이 사라져 조회로는 알 수 없다

네 낱말 — binlog · Connect · Debezium · 소스/싱크

01

binlog

모든 변경을 순서대로
기록하는 로그(ROW)

02

Kafka Connect

커넥터를 부리는
실행 프레임워크

03

Debezium

Connect 위의
CDC 소스 커넥터 모음

04

소스 / 싱크

가져오면 소스(오늘)
내보내면 싱크(6-5)

binlog_format 이 ROW 여야 한다 — STATEMENT·MIXED 면 실행된 SQL 문만 남아 행 단위 값을 복원할 수 없다

커넥터 등록 뒤 두 단계 — 스냅숏 → 스트리밍

1 **① 스냅숏 — 기존 데이터 전체를 한 번 읽는다**
op = "r"(read) 이벤트로 나오고, 이 동안 binlog 위치를 기억해 둔다

2 **② 스트리밍 — 기억한 위치부터 binlog 를 따라간다**
변경이 op = c / u / d 로 흘러나온다

3 **스냅숏 없이 스트리밍만 하면**
「지금부터의 변경」만 알고 기존 데이터는 모른다 — 그래서 처음 한 번은 전체를 읽는다

옛 데이터부터 나오는 이유

스냅숏

op = r

변경 이벤트 — op 하나에 before·after 한 쌍

op	뜻	before	after
r	스냅숏 읽기	null	현재 행
c	INSERT	null	넣은 행
u	UPDATE	바뀌기 전 행	바뀐 뒤 행
d	DELETE	지워진 행	null

before 와 after 를 함께 주므로 「무엇이 어떻게 바뀌었는지」를 받는 쪽이 안다 — 감사 로그·조건 처리가 가능

5. 토픽 이름

토픽 이름 규칙 — prefix . DB . 테이블

topic.prefix	데이터베이스	테이블	토픽 이름
"shop"	shop	orders	shop.shop.orders

테이블마다 토픽이 하나씩 생긴다 — 테이블이 100개면 토픽도 100개

커넥터 등록 때 topic.prefix 를 정한다 — 그 뒤에 DB 이름과 테이블 이름이 점으로 이어진다

6. 멱등

받는 쪽은 멱등해야 한다 — 최소 한 번 전달

1. **CDC 는 최소 한 번(at-least-once) 전달**
   커넥터가 재시작하면 같은 이벤트가 다시 올 수 있다

2. **같은 이벤트를 두 번 받아도 결과가 같아야**
   4-3 의 MERGE INTO — 키가 있으면 갱신, 없으면 삽입

3. **DELETE 도 MERGE 로**
   WHEN MATCHED AND op='d' THEN DELETE 를 붙여 처리

6-3 에서 본 그대로

유실을 막으면

중복이 생긴다

정리

이 1시간에서 꼭 가져갈 것

1. 폴링은 DB 에 부담을 주고 DELETE 를 놓친다 — CDC 는 둘 다 해결
2. binlog_format 이 ROW 여야 행 단위 값을 얻는다
3. 커넥터는 스냅숏(op=r) 뒤 스트리밍(c/u/d)으로 이어진다
4. before/after 를 함께 주므로 「무엇이 어떻게 바뀌었는지」를 안다
5. 토픽 이름은 prefix.DB.테이블 — 테이블마다 하나씩
6. CDC 는 최소 한 번 전달 — 받는 쪽이 멱등해야 한다

6-4 · 이론 1H 마무리

다음은 커넥터 구축·이벤트 분석 실습 (2.5H)

compose 에 mysql·connect 를 추가하고 커넥터를 등록해, 네 가지 op 를 전부 눈으로 확인합니다

- ✓ **log_bin=ON · binlog_format=ROW 확인**
- ✓ **REST API 로 mysql-cdc 커넥터 등록**
- ✓ **스냅숏(op=r) → INSERT·UPDATE·DELETE 관찰**
- ✓ **cdc_reader.py 로 사람이 읽기 좋게**

준비할 것

6-2 의 compose 스택(kafka) 실행 상태